Contents

1 Basic 1
- 1.1 Default . . . 1
- 1.2 builtin . . . 1
- 1.3 random . . . 1
- 1.4 run.bat . . . 1
- 1.5 run.sh . . . 1

2 Data Structure 1
- 2.1 Segment Tree . . . 1
- 2.2 segment Tree2 . . . 2
- 2.3 BIT . . . 2
- 2.4 離散化 . . . 2
- 2.5 DSU . . . 3
- 2.6 Kruskal . . . 3

3 Graph 3
- 3.1 BCC . . . 3
- 3.2 SCC . . . 3
- 3.3 支配樹 . . . 3
- 3.4 最大團 . . . 4
- 3.5 Topo . . . 4
- 3.6 找環 . . . 4
- 3.7 無向圖環路徑 . . . 5
- 3.8 有向圖環路徑 . . . 5
- 3.9 無向圖歐拉路徑 . . . 5
- 3.10SPFA . . . 5
- 3.1101bfs . . . 5
- 3.12dijk . . . 5
- 3.13flow/dinic . . . 5
- 3.14flow/ISFP . . . 6
- 3.15最小花費最大流 . . . 6
- 3.16km/二分圖最大權匹配 . . . 6
- 3.17二分圖最大匹配 . . . 7
- 3.18結論 . . . 7

4 geometry 7
- 4.1 Point . . . 7
- 4.2 Line . . . 7
- 4.3 Circle . . . 8
- 4.4 點線距離 . . . 8
- 4.5 線段相交 . . . 8
- 4.6 inPolygon . . . 8
- 4.7 ConvexHull . . . 8
- 4.8 最小圓覆蓋 . . . 8
- 4.9 旋轉卡尺 . . . 8

5 Tree 8
- 5.1 is_anc . . . 8
- 5.2 LCA . . . 8
- 5.3 倍增 . . . 8
- 5.4 tree_flaten . . . 9

6 DP 9
- 6.1 LIS . . . 9
- 6.2 LCS . . . 9
- 6.3 拆分 . . . 9
- 6.4 編輯距離 . . . 9
- 6.5 SOSDP . . . 9
- 6.6 p_median . . . 9

7 search 9
- 7.1 binary search . . . 9
- 7.2 三分搜 . . . 9

8 Math 9
- 8.1 質數篩 . . . 9
- 8.2 fpow . . . 10
- 8.3 exgcd . . . 10
- 8.4 組合數 . . . 10
- 8.5 josephus . . . 10
- 8.6 josephus2 . . . 10
- 8.7 FFT . . . 10
- 8.8 Matrices_fpow . . . 10
- 8.9 Miller-Rabin . . . 11
- 8.10pollard rho . . . 11

9 String 11
- 9.1 KMP . . . 11
- 9.2 AC 自動機 . . . 11
- 9.3 minRotation . . . 11
- 9.4 迴文樹 . . . 12
- 9.5 Rolling Hash . . . 12
- 9.6 SAIS . . . 12
- 9.7 trie . . . 13
- 9.8 z_value . . . 13
- 9.9 馬拉車 . . . 13

10 好東西 13
- 10.1大悲咒 . . . 13
- 10.2心經 . . . 13

1 Basic

1.1 Default

```cpp
#include<bits/stdc++.h>
using namespace std;
#define int long long
#define Harry55 ios_base::sync_with_stdio(0), cin.tie
    (0), cin.exceptions(cin.failbit);
#define pii pair<int, int>
#define vi vector<int>
#define ff first
#define ss second
#define pb push_back
#define all(x) (x).begin(), (x).end()

void solve(){
}

signed main(){
    int t = 1;
    //cin >> t;
    while(t--){
        solve();
}}
```

1.2 builtin

```cpp
__builtin_popcountll //二進位有幾個1(記得這是long long)
__builtin_clzll //左起第一個1之前0的個數
__builtin_parityll //1的個數的奇偶性
__builtin_mul_overflow(a,b,&h) //a*b是否溢位,h = a * b
__builtin_add_overflow(a,b,&h)
```

1.3 random

```cpp
mt19937 mt(chrono::steady_clock::now().time_since_epoch
    ().count());
//mt19937_64 mt() -> return randnum
int randint(int l, int r){
    uniform_int_distribution<> dis(l, r); return dis(mt
        );
}
```

1.4 run.bat

```bat
@echo off
g++ ac.cpp -o ac.exe
g++ wa.cpp -o wa.exe
set /a num=1
:loop
    echo %num%
    python gen.py > input
    ac.exe < input > ac
    wa.exe < input > wa
    fc ac wa
    set /a num=num+1
if not errorlevel 1 goto loop
```

1.5 run.sh

```sh
set -e
for ((i=0;;i++))
do
    echo "$i"
    python gen.py > in
    ./ac < in > ac.out
    ./wa < in > wa.out
    diff ac.out wa.out || break
done
```

2 Data Structure

2.1 Segment Tree

```cpp
struct seg_tree{ //0-base 左閉右閉
    #define cl(x) (x << 1)
    #define cr(x) (x << 1) | 1
    vector<int> seg, arr;
    int n;
    void init(int _n, vector<int> &_arr){
        seg = vector<int>(4 * _n + 5);
```

```
        arr = _arr;
        n = _n;
        build(1, 0, _n - 1);
    }
    void pull(int id){
        // 修改條件
        seg[id] = min(seg[cl(id)], seg[cr(id)]);
    }
    void build(int id, int l, int r){
        if(l == r){
            seg[id] = arr[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(cl(id), l, mid);
        build(cr(id), mid + 1, r);
        pull(id);
    }
    void upd(int id, int l, int r, int x, int v){
        if(l == r){
            seg[id] = v;
            return;
        }
        int mid = (l + r) >> 1;
        if(x <= mid) upd(cl(id), l, mid, x, v);
        else upd(cr(id), mid + 1, r, x, v);
        pull(id);
    }
    int query(int id, int l, int r, int sl, int sr){
        if(sl <= l && r <= sr){
            return seg[id];
        }
        int mid = (l + r) >> 1, res = INF;
        if(sl <= mid) res = min(res, query(cl(id), l,
            mid, sl, sr));
        if(mid < sr) res = min(res, query(cr(id), mid +
             1, r, sl, sr));
        return res;
    }
    int find_first(int id, int l, int r, int sl, int sr
        , int k){//二分搜
        if(l > sr || r < sl || seg[id] < k) return -1;
        if(l == r) return l;
        int mid = (l + r) >> 1;
        int res = find_first(cl(id), l, mid, sl, sr, k)
            ;
        if(res == -1) res = find_first(cr(id), mid + 1,
             r, sl, sr, k);
        return res;
    }
    int find_first(int l, int r, int k) {
        return find_first(1, 0, n - 1, l, r, k);
    }
    int query(int sl, int sr){
        return query(1, 0, n - 1, sl, sr);
    }
    void upd(int x, int v){
        upd(1, 0, n - 1, x, v);
}};
```

2.2 segment Tree2

```
struct seg_tree{
    #define cl(x) (x << 1)
    #define cr(x) (x << 1) | 1
    vector<int> seg, arr, tag;
    int n;
    void init(int _n, const vector<int> &_arr){
        seg = vector<int>(4 * _n + 5);
        tag = vector<int>(4 * _n + 5);
        arr = _arr;
        n = _n;
        build(1, 0, _n - 1);
    }
    void pull(int id, int l, int r){
        int mid = (l+r)>>1;
        push(cl(id), l, mid);
        push(cr(id), mid + 1, r);
        seg[id] = seg[cl(id)] + seg[cr(id)];
    }
    void push(int id, int l, int r){
        if(tag[id]) {
            seg[id] += tag[id] * (r - l + 1);
            if(l != r) {
                tag[cl(id)] += tag[id];
                tag[cr(id)] += tag[id];
            }
            tag[id] = 0;
        }
    }
    void build(int id, int l, int r){
        if(l == r){
            seg[id] = arr[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(cl(id), l, mid);
        build(cr(id), mid + 1, r);
        pull(id, l, r);
    }
    void upd(int id, int l, int r, int sl, int sr, int
        v){
        push(id, l, r);
        if(sl <= l && r <= sr){
            tag[id] += v;
            return;
        }
        int mid = (l + r) >> 1;
        if(sl <= mid) upd(cl(id), l, mid, sl, sr, v);
        if(sr > mid) upd(cr(id), mid + 1, r, sl, sr, v)
            ;
        pull(id, l, r);
    }
    int query(int id, int l, int r, int sl, int sr){
        push(id, l, r);
        if(sl <= l && r <= sr){
            return seg[id];
        }
        int mid = (l + r) >> 1, res = 0;
        if(sl <= mid) res += query(cl(id), l, mid, sl,
            sr);
        if(mid < sr) res += query(cr(id), mid + 1, r,
            sl, sr);
        return res;
    }
    int query(int sl, int sr){
        return query(1, 0, n - 1, sl, sr);
    }
    void upd(int sl, int sr, int v) {
        upd(1, 0, n - 1, sl, sr, v);
}};
```

2.3 BIT

```
struct BIT{
    int n;
    vector<int> bit;
    #define lowbit(x) (x & -x)
    void init(int _n){
        n = _n + 2;
        bit = vector<int>(n, 0);
    }
    int _query(int x){
        int ans = 0;
        for( ; x > 0 ; x -= lowbit(x)) ans += bit[x];
        return ans;
    }
    void _upd(int x, int v){
        for(; x <= n ; x += lowbit(x)) bit[x] += v;
    }
    void upd(int x, int v) {_upd(x + 1, v);}
    int query(int x) {return _query(x + 1);}
    int query(int l, int r) {return (query(r) - query(l
         - 1));}
};
//搭配字元修改 + Hash時的操作
void update(int pos, int ch){
    BIT.update(pos, (ch - s[pos]) * p[pos] % MOD);
    revBIT.update(n-pos+1, (ch-s[pos]) * p[n-pos+1] %
        MOD);
}
```

2.4 離散化

```
vector<int> tmp(arr);
sort(all(tmp));
tmp.erase(unique(all(tmp)), tmp.end());
for(int i = 0 ; i < n ; i++) arr[i] = lower_bound(all(
    tmp), arr[i]) - tmp.begin();
```

2.5 DSU

```
struct DSU {
    vector<int> f, sz;
    int cnt;
    DSU(int n) : f(n), sz(n), cnt(n) {
        for (int i = 0; i < n; i++) {
            f[i] = i;
            sz[i] = 1;
        }
    }
    int find(int x) {
        if (x == f[x]) return x;
        f[x] = find(f[x]);
        return f[x];
    }
    int merge(int x, int y) {
        x = find(x), y = find(y);
        if (x == y) return 0;
        if (sz[x] < sz[y])
            swap(x, y);
        sz[x] += sz[y];
        f[y] = x;
        cnt--;
        return 1;
    }
    bool same(int a, int b) { return (find(a) == find(b
        )); }
    int group() {return cnt;}
};
```

2.6 Kruskal

```
struct Edge {
    int u, v, w;
    bool operator<(const Edge& other) const {return w <
         other.w;}
};
int kruskal(int n, vector<Edge>& edges) {
    dsu.init(n);
    sort(edges.begin(), edges.end());
    int mst_weight = 0;
    int edge_cnt = 0;
    for (auto e : edges) {
        if (dsu.merge(e.u, e.v)) {
            mst_weight += e.w;
            edge_cnt++;
            if (edge_cnt == n - 1) break;
        }
    }
    if (edge_cnt < n - 1) return -1;
    return mst_weight;
}
```

3 Graph

3.1 BCC

```
struct BccVertex {
  int n,nScc,step,dfn[MXN],low[MXN];
  vector<int> E[MXN],sccv[MXN];
  int top,stk[MXN];
  void init(int _n) {
    n = _n; nScc = step = 0;
    for (int i=0; i<n; i++) E[i].clear();
  }
  void addEdge(int u, int v)
  { E[u].PB(v); E[v].PB(u); }
  void DFS(int u, int f) {
    dfn[u] = low[u] = step++;
    stk[top++] = u;
    for (auto v:E[u]) {
      if (v == f) continue;
      if (dfn[v] == -1) {
        DFS(v,u);
        low[u] = min(low[u], low[v]);
        if (low[v] >= dfn[u]) {
          int z;
          sccv[nScc].clear();
          do {
            z = stk[--top];
            sccv[nScc].PB(z);
          } while (z != v);
          sccv[nScc++].PB(u);
        }
      }else
        low[u] = min(low[u],dfn[v]);
  } }
  vector<vector<int>> solve() {
    vector<vector<int>> res;
    for (int i=0; i<n; i++)
      dfn[i] = low[i] = -1;
    for (int i=0; i<n; i++)
      if (dfn[i] == -1) {
        top = 0;
        DFS(i,i);
      }
    REP(i,nScc) res.PB(sccv[i]);
    return res;
  }
}graph;
//vector size = 2的為橋
//vector size > 2的為點雙連通分量
```

3.2 SCC

```
struct Scc{
//bln:在哪個分量 E原圖
  int n, nScc, vst[MXN], bln[MXN];
  vector<int> E[MXN], rE[MXN], vec;
  void init(int _n){
    n = _n;
    for (int i=0; i<= n; i++)
      E[i].clear(), rE[i].clear();
  }
  void addEdge(int u, int v){
    E[u].PB(v); rE[v].PB(u);
  }
  void DFS(int u){
    vst[u]=1;
    for (auto v : E[u]) if (!vst[v]) DFS(v);
    vec.PB(u);
  }
  void rDFS(int u){
    vst[u] = 1; bln[u] = nScc;
    for (auto v : rE[u]) if (!vst[v]) rDFS(v);
  }
  void solve(){
    nScc = 0;
    vec.clear();
    fill(vst, vst+n+1, 0);
    for (int i=0; i<n; i++)
      if (!vst[i]) DFS(i);
    reverse(vec.begin(),vec.end());
    fill(vst, vst+n+1, 0);
    for (auto v : vec)
      if (!vst[v]){
        rDFS(v); nScc++;
      }
  }
};
```

3.3 支配樹

```
//支配點
//最後 idom[u] 為點 u 的支配點
struct DominatorTree{ // O(N)
#define REP(i,s,e) for(int i=(s);i<=(e);i++)
#define REPD(i,s,e) for(int i=(s);i>=(e);i--)
  int n , s;
  vector< int > g[ MAXN ] , pred[ MAXN ];
  vector< int > cov[ MAXN ];
  int dfn[ MAXN ] , nfd[ MAXN ] , ts;
  int par[ MAXN ]; //idom[u] s到u的最後一個必經點
  int sdom[ MAXN ] , idom[ MAXN ];
  int mom[ MAXN ] , mn[ MAXN ];
  inline bool cmp( int u , int v )
```

```
  { return dfn[ u ] < dfn[ v ]; }
  int eval( int u ){
    if( mom[ u ] == u ) return u;
    int res = eval( mom[ u ] );
    if(cmp( sdom[ mn[ mom[ u ] ] ] , sdom[ mn[ u ] ] ))
      mn[ u ] = mn[ mom[ u ] ];
    return mom[ u ] = res;
  }
  void init( int _n , int _s ){
    ts = 0; n = _n; s = _s;
    REP( i, 1, n ) g[ i ].clear(), pred[ i ].clear();
  }
  void addEdge( int u , int v ){
    g[ u ].push_back( v );
    pred[ v ].push_back( u );
  }
  void dfs( int u ){
    ts++;
    dfn[ u ] = ts;
    nfd[ ts ] = u;
    for( int v : g[ u ] ) if( dfn[ v ] == 0 ){
      par[ v ] = u;
      dfs( v );
  } }
  void build(){
    REP( i , 1 , n ){
      dfn[ i ] = nfd[ i ] = 0;
      cov[ i ].clear();
      mom[ i ] = mn[ i ] = sdom[ i ] = i;
    }
    dfs( s );
    REPD( i , n , 2 ){
      int u = nfd[ i ];
      if( u == 0 ) continue ;
      for( int v : pred[ u ] ) if( dfn[ v ] ){
        eval( v );
        if( cmp( sdom[ mn[ v ] ] , sdom[ u ] ) )
          sdom[ u ] = sdom[ mn[ v ] ];
      }
      cov[ sdom[ u ] ].push_back( u );
      mom[ u ] = par[ u ];
      for( int w : cov[ par[ u ] ] ){
        eval( w );
        if( cmp( sdom[ mn[ w ] ] , par[ u ] ) )
          idom[ w ] = mn[ w ];
        else idom[ w ] = par[ u ];
      }
      cov[ par[ u ] ].clear();
    }
    REP( i , 2 , n ){
      int u = nfd[ i ];
      if( u == 0 ) continue ;
      if( idom[ u ] != sdom[ u ] )
        idom[ u ] = idom[ idom[ u ] ];
} } }domT;
```

3.4 最大團

```
//最大團定義:團內每個點都有一條邊連接到其他點
//O(1.4422^n)
#define N 111
struct MaxClique{ // 0-base
  typedef bitset<N> Int;
  Int linkto[N] , v[N];
  int n;
  void init(int _n){
    n = _n;
    for(int i = 0 ; i < n ; i ++){
      linkto[i].reset(); v[i].reset();
  } }
  void addEdge(int a , int b)
  { v[a][b] = v[b][a] = 1; }
  int popcount(const Int& val)
  { return val.count(); }
  int lowbit(const Int& val)
  { return val._Find_first(); }
  int ans , stk[N];
  int id[N] , di[N] , deg[N];
  Int cans;
  void maxclique(int elem_num, Int candi){
    if(elem_num > ans){
      ans = elem_num; cans.reset();
      for(int i = 0 ; i < elem_num ; i ++)
        cans[id[stk[i]]] = 1;
    }
    int potential = elem_num + popcount(candi);
    if(potential <= ans) return;
    int pivot = lowbit(candi);
    Int smaller_candi = candi & (~linkto[pivot]);
    while(smaller_candi.count() && potential > ans){
      int next = lowbit(smaller_candi);
      candi[next] = !candi[next];
      smaller_candi[next] = !smaller_candi[next];
      potential --;
      if(next == pivot || (smaller_candi & linkto[next
          ]).count()){
        stk[elem_num] = next;
        maxclique(elem_num + 1, candi & linkto[next]);
  } } }
  int solve(){
    for(int i = 0 ; i < n ; i ++){
      id[i] = i; deg[i] = v[i].count();
    }
    sort(id , id + n , [&](int id1, int id2){
          return deg[id1] > deg[id2]; });
    for(int i = 0 ; i < n ; i ++) di[id[i]] = i;
    for(int i = 0 ; i < n ; i ++)
      for(int j = 0 ; j < n ; j ++)
        if(v[i][j]) linkto[di[i]][di[j]] = 1;
    Int cand; cand.reset();
    for(int i = 0 ; i < n ; i ++) cand[i] = 1;
    ans = 1;
    cans.reset(); cans[0] = 1;
    maxclique(0, cand);
    return ans;
} }solver;
/*獨立集 與 團 的概念完全相反 最大獨立集去建反圖之後，
    求最大團即可得到答案
獨立集定義: 獨立集內的任意兩個點之間都?有?相?
*/
```

3.5 Topo

```
vector<int> edge[N], deg(N, 0), ans;
bool topo(int n){
    ans.clear();
    queue<int> q;
    for(int i = 1 ; i <= n ; i++) if(deg[i] == 0) q.
        push(i);
    while(!q.empty()){
        int u = q.front(); q.pop();
        ans.push_back(u);
        for(auto i : edge[u]){
            if(--deg[i] == 0) q.push(i);
        }
    }
    return n == ans.size();
}
```

3.6 找環

```
int vis[N] = {0};
bool dfs(int u) {
    if (vis[u] == 1) return true;
    if (vis[u] == 2) return false;
    vis[u] = 1;
    for (auto v : edge[u]) {
        if (dfs(v)) {
            return true;
        }
    }
    vis[u] = 2;
    return false;
}//有向圖找環
bool dfs(int x, int v) {
    if (vis[x]) return true;
    vis[x] = 1;
    for (auto i : edge[x]) {
        if (i == v) continue;
        if (dfs(i, x)) return true;
    }
    return false;
}//無向圖找環
```

3.7 無向圖環路徑

```
int vis[], suc[];
int dfs(int x, int v) { //v -> x 是 tree edge
    if (vis[x]) return x;
    suc[x] = v; //紀錄 x 的父節點 v
    vis[x] = 1;
    for (auto i : edge[x]) {
        if (i == v) continue;
        int tmp = dfs(i, x); // x 的祖先，
        if (tmp) {
            int now = x;
            vector<int> ans;
            ans.push_back(i);
            while (now != tmp) { //找從x到tmp的路徑
                ans.push_back(now);
                now = suc[now];
            }
            ans.push_back(i);
            for (auto v : ans) cout << v << " ";
            cout << "\n";
            exit(0);
        }
    }
    return 0;
}
```

3.8 有向圖環路徑

```
void dfs(int u){
    vis[u] = 1;
    for (auto i : edge[u]) {
        if (vis[i] == 1) {
            vector<int> ans;
            ans.push_back(i);
            int now = u;
            while (now != i) {
                ans.push_back(now);
                now = suc[now];
            }
            ans.push_back(i);
            reverse(ans.begin(), ans.end());
            cout << ans.size() << "\n";
            for (auto i : ans) cout << i << " ";
            exit(0);
        }
        if (vis[i] == 0) {
            suc[i] = u;
            dfs(i);
        }
    }
    vis[u] = 2;
}
```

3.9 無向圖歐拉路徑

```
void dfs(int u, vector<pii> edge[], vector<int> &vis){
    while(!edge[u].empty()){
        auto [v, idx] = edge[u].back(); edge[u].
            pop_back();
        if(vis[idx]) continue;
        vis[idx] = 1;
        dfs(v, edge, vis);
        epath.push_back(idx); // edge path
    }
    path.push_back(u); // node path
}//reverse
```

3.10 SPFA

```
#define MXN 200005
struct SPFA{
  int n;
  LL inq[MXN], len[MXN];
  vector<LL> dis;
  vector<pair<int, LL>> edge[MXN];
  void init(int _n){
    n = _n;
    dis.clear(); dis.resize(n, 1e18);
    for(int i = 0; i < n; i++){
      edge[i].clear();
      inq[i] = len[i] = 0;
    } }
  void addEdge(int u, int v, LL w){
    edge[u].push_back({v, w});
  }
  vector<LL> solve(int st = 0){
    deque<int> dq; //return {-1} if has negative cycle
    dq.push_back(st); //otherwise return dis from st
    inq[st] = 1; dis[st] = 0;
    while(!dq.empty()){
      int u = dq.front();  dq.pop_front();
      inq[u] = 0;
      for(auto [to, d] : edge[u]){
        if(dis[to] > d+dis[u]){
          dis[to] = d+dis[u];
          len[to] = len[u]+1;
          if(len[to] > n) return {-1};
          if(inq[to]) continue;
          (!dq.empty()&&dis[dq.front()] > dis[to]?
              dq.push_front(to) : dq.push_back(to));
          inq[to] = 1;
    } } }
    return dis;
} }spfa;
```

3.11 01bfs

```
void bfs_01(int s, int n) {
    for(int i = 1; i <= n; i++) dis[i] = INF;
    deque<int> dq;
    dis[s] = 0;
    dq.push_front(s);
    while(!dq.empty()) {
        int u = dq.front();dq.pop_front();
        for(auto [v, w] : edge[u]) {
            if (dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                if (w == 0) dq.push_front(v);
                else dq.push_back(v);
            }
        }
    }
}
```

3.12 dijk

```
const int N = 1e5 + 5;
const int INF = 1e18;
vector<pii> edge[N];
vector<int> dis(N, INF);
void dijk(int s){
    dis[s] = 0;
    vector<int> vis(N, 0);
    priority_queue<pii,vector<pii>,greater<pii>> pq;
    pq.push({0, s});
    while(!pq.empty()){
        int u = pq.top().ss; pq.pop();
        if(vis[u]) continue; vis[u] = 1;
        for(auto [v, w] : edge[u]){
            if(dis[u] + w < dis[v]) {
                dis[v] = dis[u] + w;
                pq.push({dis[v], v});
            }
        }
    }
}
```

3.13 flow/dinic

```
#define PB push_back
#define SZ(x) ((int)(x).size())
const int MXN = 200005;   // 依題目調整，要 >= 點數

struct Dinic {
    struct Edge { int v; LL f; int re; };
    int n, s, t, level[MXN], iter[MXN];
    vector<Edge> E[MXN];
    void init(int _n, int _s, int _t) {
        n = _n; s = _s; t = _t;
        for (int i = 0; i < n; i++) E[i].clear();
    }
    void add_edge(int u, int v, LL f) {
        int a = SZ(E[u]), b = SZ(E[v]) + (u == v);
```

```
        E[u].PB({v, f, b});
        E[v].PB({u, 0, a});
    }
    bool BFS() {
        for (int i = 0; i < n; i++) level[i] = -1;
        queue<int> q; q.push(s); level[s] = 0;
        while (!q.empty()) {
            int u = q.front(); q.pop();
            for (auto &e : E[u])
                if (e.f > 0 && level[e.v] == -1) {
                    level[e.v] = level[u] + 1;
                    q.push(e.v);
                }
        }
        return level[t] != -1;
    }
    LL DFS(int u, LL nf) {
        if (u == t) return nf;
        LL res = 0;
        for (int &i = iter[u]; i < SZ(E[u]); i++) {
            // current arc
            Edge &e = E[u][i];
            if (e.f > 0 && level[e.v] == level[u] + 1)
                {
                LL tf = DFS(e.v, min(nf, e.f));
                res += tf; nf -= tf;
                e.f -= tf; E[e.v][e.re].f += tf;
                if (nf == 0) return res;
            }
        }
        if (!res) level[u] = -1;
        return res;
    }
    LL flow() {
        LL res = 0;
        while (BFS()) {
            fill(iter, iter + n, 0);
            res += DFS(s, LLONG_MAX);
        }
        return res;
    }//最大匹配 = 最小點覆蓋
} flow; //最小邊覆蓋 = 最大獨立集
/*
index[u].push_back({v, flow.E[u].size()});
for (int i = 1; i <= left_n; i++) { // 遍歷所有左邊的節
    點
    for (pair<int, int> j : index[i]) {
        if (flow.E[i][j.second - 1].f == 0)
            cout << i << ' ' << j.first << '\n';
    }
}
*/
```

3.14 flow/ISFP

```
#define SZ(x) ((int)(x).size())

struct ISAP {
    static const int MAXV = 200010;
    struct Edge { int v; LL c; int r; };
    int n, s, t;
    vector<Edge> G[MAXV];
    int iter[MAXV], d[MAXV], gap[MAXV + 1];

    void init(int _n, int _s, int _t) {        // 點編
        號 0 ~ n-1，自己指定 s、t
        n = _n; s = _s; t = _t;
        for (int i = 0; i < n; i++) G[i].clear();
    }
    void addEdge(int u, int v, LL c) {
        int a = SZ(G[u]), b = SZ(G[v]) + (u == v);
        G[u].push_back({v, c, b});
        G[v].push_back({u, 0, a});
    }
    LL dfs(int p, LL flow) {
        if (p == t) return flow;
        LL used = 0;
        for (int &i = iter[p]; i < SZ(G[p]); i++) {
            Edge &e = G[p][i];
            if (e.c > 0 && d[p] == d[e.v] + 1) {
                LL f = dfs(e.v, min(flow - used, e.c));
                e.c -= f; G[e.v][e.r].c += f; used += f
                    ;
                if (used == flow || d[s] >= n) return
                    used;
            }
        }
        if (--gap[d[p]] == 0) d[s] = n;        // gap
            優化
        else { d[p]++; iter[p] = 0; ++gap[d[p]]; }
        return used;
    }
    LL solve() {
        fill(iter, iter + n, 0); fill(d, d + n, 0);
            fill(gap, gap + n + 1, 0);
        gap[0] = n;
        LL res = 0;
        while (d[s] < n) res += dfs(s, LLONG_MAX);
        return res;
    }
} flow;
//flow.init(n, s, t);           // 點 0 ~ n-1
//flow.addEdge(u, v, cap);
//LL ans = flow.solve();
```

3.15 最小花費最大流

```
struct zkwflow{
  static const int maxN=10000;
  struct Edge{ int v,f,re; ll w;};
  int n,s,t,ptr[maxN]; bool vis[maxN]; ll dis[maxN];
  vector<Edge> E[maxN];
  void init(int _n,int _s,int _t){
    n=_n,s=_s,t=_t;
    for(int i=0;i<n;i++) E[i].clear();
  }
  void addEdge(int u,int v,int f,ll w){
    E[u].push_back({v,f,(int)E[v].size(),w});
    E[v].push_back({u,0,(int)E[u].size()-1,-w});
  }
  bool SPFA(){
    fill_n(dis,n,LLONG_MAX); fill_n(vis,n,false);
    queue<int> q; q.push(s); dis[s]=0;
    while (!q.empty()){
      int u=q.front(); q.pop(); vis[u]=false;
      for(auto &it:E[u]){
        if(it.f>0&&dis[it.v]>dis[u]+it.w){
          dis[it.v]=dis[u]+it.w;
          if(!vis[it.v]){
            vis[it.v]=true; q.push(it.v);
  } } } }
    return dis[t]!=LLONG_MAX;
  }
  int DFS(int u,int nf){
    if(u==t) return nf;
    int res=0; vis[u]=true;
    for(int &i=ptr[u];i<(int)E[u].size();i++){
      auto &it=E[u][i];
      if(it.f>0&&dis[it.v]==dis[u]+it.w&&!vis[it.v]){
        int tf=DFS(it.v,min(nf,it.f));
        res+=tf,nf-=tf,it.f-=tf;
        E[it.v][it.re].f+=tf;
        if(nf==0){ vis[u]=false; break; }
      }
    }
    return res;
  }
  pair<int,ll> flow(){
    int flow=0; ll cost=0;
    while (SPFA()){
      fill_n(ptr,n,0);
      int f=DFS(s,INT_MAX);
      flow+=f; cost+=dis[t]*f;
    }
    return{ flow,cost };
  } // reset: do nothing
} flow;
```

3.16 km/二分圖最大權匹配

```
  struct KM{ // max weight, for min negate the weights
  int n, mx[MXN], my[MXN], pa[MXN];
  ll g[MXN][MXN], lx[MXN], ly[MXN], sy[MXN];
```

```
  bool vx[MXN], vy[MXN];
  void init(int _n) { // 1-based, N個節點
    n = _n;
    for(int i=1; i<=n; i++) fill(g[i], g[i]+n+1, 0);
  }
  void addEdge(int x, int y, ll w) {g[x][y] = w;} //左
      邊的集合節點x連邊右邊集合節點y權重為w
  void augment(int y) {
    for(int x, z; y; y = z)
      x=pa[y], z=mx[x], my[y]=x, mx[x]=y;
  }
  void bfs(int st) {
    for(int i=1; i<=n; ++i) sy[i]=INF, vx[i]=vy[i]=0;
    queue<int> q; q.push(st);
    for(;;) {
      while(q.size()) {
        int x=q.front(); q.pop(); vx[x]=1;
        for(int y=1; y<=n; ++y) if(!vy[y]){
          ll t = lx[x]+ly[y]-g[x][y];
          if(t==0){
            pa[y]=x;
            if(!my[y]){augment(y);return;}
            vy[y]=1, q.push(my[y]);
          }else if(sy[y]>t) pa[y]=x,sy[y]=t;
      } }
      ll cut = INF;
      for(int y=1; y<=n; ++y)
        if(!vy[y]&&cut>sy[y]) cut=sy[y];
      for(int j=1; j<=n; ++j){
        if(vx[j]) lx[j] -= cut;
        if(vy[j]) ly[j] += cut;
        else sy[j] -= cut;
      }
      for(int y=1; y<=n; ++y) if(!vy[y]&&sy[y]==0){
        if(!my[y]){augment(y);return;}
        vy[y]=1, q.push(my[y]);
  } } }
  ll solve(){ // 回傳值為完美匹配下的最大總權重
    fill(mx, mx+n+1, 0); fill(my, my+n+1, 0);
    fill(ly, ly+n+1, 0); fill(lx, lx+n+1, -INF);
    for(int x=1; x<=n; ++x) for(int y=1; y<=n; ++y) //
        1-base
      lx[x] = max(lx[x], g[x][y]);
    for(int x=1; x<=n; ++x) bfs(x);
    ll ans = 0;
    for(int y=1; y<=n; ++y) ans += g[my[y]][y];
    return ans;
} }graph;
```

3.17 二分圖最大匹配

```
bool dfs(int u) {  // 找最大匹配
    for (int &v : g[u]) {
        if (vis[v]) continue; // 有連通且未拜訪
        vis[v] = true;
        if (S[v] == -1 || dfs(S[v])) {
            S[v] = u;    // 紀錄匹配
            return true; // 反轉匹配邊以及未匹配邊的狀
                態
        }
    }
    return false;
}
ans = 0; // 記得每次使用需清空 vis，S 初始化為 -1
for (int i = 1; i <= n; i++) {  // 跑左集合
    memset(vis, 0, sizeof(vis));
    if (dfs(i)) ans++;
}// 找二分圖最大匹配
```

3.18 結論

- 2-SAT :
  $(a_i \vee a_j) = true \ \forall (i,j)$
  對於任意限制 $(x \vee y)$
  建兩條有向邊 (要多編號 $\neg x$)
  $x \to \neg y$ and $y \to \neg x$
  跑 scc
  scc.bln[x] < scc.bln[$\neg x$] $\Leftrightarrow$ x is true
  scc.bln[$\neg x$] < scc.bln[x] $\Leftrightarrow$ x is false
  $\exists x$ which scc.bln[x] == scc.bln[$\neg x$] $\Leftrightarrow$ 無解

- 差分約束:
  n 個變數及 m 個約束條件
  求滿足所有 $x_j - x_i \le b_k \ \ (i,j \in [1,n], k \in [1,m])$
  的一組 $x_1 \ \dots \ x_n$
  可轉成 $x_j - x_i \le b_k \ \to \ x_j \le x_i + b_k$
  結論就是使得所有 x_j 變小以滿足上式
  建邊跑 SPFA/Bellman
  要多建起點 s 連到所有 i 且邊權 0, dis[s] = 0
  有負環則無解，否則起點到所有 i 的距離為一組解
  $x_j - x_i \le k \Rightarrow$ addEdge $i \xrightarrow{k} j$
  $x_j - x_i \ge k \Rightarrow$ addEdge $j \xrightarrow{-k} i$
  $x_j = x_i \Rightarrow$ addEdge $i \xrightarrow{0} j$ and $j \xrightarrow{0} i$

4 geometry

4.1 Point

```
using ld = long double;
template<class T>
struct pt{
  T x,y;
  pt(T _x,T _y):x(_x),y(_y){}
  pt():x(0),y(0){}

  pt operator * (T  c){ return pt(x*c,y*c);}
  pt operator / (T  c){ return pt(x/c,y/c);}
  pt operator + (pt a){ return pt(x+a.x,y+a.y);}
  pt operator - (pt a){ return pt(x-a.x,y-a.y);}
  T  operator * (pt a){ return x*a.x + y*a.y;}
  T  operator ^ (pt a){ return x*a.y - y*a.x;}
  bool operator < (pt a) const { return x < a.x || (x
      == a.x && y < a.y);};
  bool operator== (pt a) const { return x == a.x and y
      == a.y;};
  friend T ori(pt a, pt b, pt c) { return (b - a) ^ (c
      - a); }
  friend T abs2(pt a) { return a * a; }
};
const ld pi = acos(-1);
const ld eps = 1e-8L;
using Pt = pt<ld>;
// using Pt = pt<int>;
int sgn(ld x) { return (x > -eps) - (x < eps); } //
    dcmp == sgn
ld abs(Pt a) { return sqrt(abs2(a)); }
ld arg(Pt x) { return atan2(x.y, x.x); }
bool argcmp(Pt a, Pt b) { // arg(a) < arg(b)
    int f = (Pt{a.y, -a.x} > Pt{} ? 1 : -1) * (a != Pt
        {});
    int g = (Pt{b.y, -b.x} > Pt{} ? 1 : -1) * (b != Pt
        {});
    return f == g ? (a ^ b) > 0 : f < g;
}
Pt unit(Pt x) { return x / abs(x); }
Pt rotate(Pt u) { // pi / 2
    return {-u.y, u.x};
}
Pt rotate(Pt u, ld a) {
    Pt v{sin(a), cos(a)};
    return {u ^ v, u * v};
}
istream &operator>>(istream &s, Pt &a) { return s >> a.
    x >> a.y; }
ostream &operator<<(ostream &s, Pt &a) { return s << "(
    " << a.x << ", " << a.y << ")";}
bool collinearity(Pt a, Pt b, Pt c) {  // 三點共線
    return ((b - a) ^ (c - a)) == 0;
}
```

4.2 Line

```
struct Line {
    Pt a, b;
    Pt dir() const { return b - a; }
};
int PtSide(Pt p, Line L) {
    // return sgn(ori(L.a, L.b, p) / abs(L.a - L.b));
    return sgn(ori(L.a, L.b, p));
}
bool PtOnSeg(Pt p, Line L) { // 共線
    return PtSide(p, L) == 0 and sgn((p - L.a) * (p - L
        .b)) <= 0;
}
```

4.3 Circle

```
struct Cir {
    Pt o;
    ld r;
};
bool disjunct(const Cir &a, const Cir &b) { //  分開 || 外切 -> 1
    return sgn(abs(a.o - b.o) - a.r - b.r) >= 0;
}
bool contain(const Cir &a, const Cir &b) {
    return sgn(a.r - b.r - abs(a.o - b.o)) >= 0;
}
```

4.4 點線距離

```
double PtSegDist(Pt p, Line l) {
    double ans = min(abs(p - l.a), abs(p - l.b));
    if (sgn(abs(l.a - l.b)) == 0)  return ans;
    if (sgn((l.a - l.b) * (p - l.b)) < 0) return ans;
    if (sgn((l.b - l.a) * (p - l.a)) < 0) return ans;
    return min(ans, abs(ori(p, l.a, l.b)) / abs(l.a - l.b));
}
double SegDist(Line l, Line m) {
    return PtSegDist({0, 0}, {l.a - m.a, l.b - m.b});
}
```

4.5 線段相交

```
bool isInter(Line l, Line m) {
    if (PtOnSeg(m.a, l) or PtOnSeg(m.b, l) or
        PtOnSeg(l.a, m) or PtOnSeg(l.b, m))
        return true;
    return PtSide(m.a, l) * PtSide(m.b, l) < 0 and
           PtSide(l.a, m) * PtSide(l.b, m) < 0;
}
```

4.6 inPolygon

```
int inPoly(Pt p, const vector<Pt> &P) {
    const int n = P.size();
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        Pt a = P[i], b = P[(i + 1) % n];
        if (PtOnSeg(p, {a, b})) return 1; // on edge
        if ((sgn(a.y - p.y) == 1) ^ (sgn(b.y - p.y) == 1))
            cnt += sgn(ori(a, b, p));
    }
    return cnt == 0 ? 0 : 2; // out, in
}
```

4.7 ConvexHull

```
vector<Pt> Hull(vector<Pt> P) {
    sort(all(P));
    P.erase(unique(all(P)), P.end());
    P.insert(P.end(), P.rbegin() + 1, P.rend());
    vector<Pt> stk;
    for (auto p : P) {
        auto it = stk.rbegin();
        //不可共線，改 < 0
        while (stk.rend() - it >= 2 and \
            ori(*next(it), *it, p) <= 0 and \
            (*next(it) < *it) == (*it < p)) {
            it++;
        }
        stk.resize(stk.rend() - it);
        stk.push_back(p);
    }
    stk.pop_back();
    return stk;
}
```

4.8 最小圓覆蓋

```
Pt Center(Pt a, Pt b, Pt c) {
    Pt x = (a + b) / 2;
    Pt y = (b + c) / 2;
    return LineInter({x, x + rotate(b - a)}, {y, y + rotate(c - b)});
}
Cir MEC(vector<Pt> P) {
    mt19937 rng(time(0));
    shuffle(all(P), rng);
    Cir C = {P[0], 0.0};
    for (int i = 0; i < P.size(); i++) {
        if (C.inside(P[i])) continue;
        C = {P[i], 0};
        for (int j = 0; j < i; j++) {
            if (C.inside(P[j])) continue;
            C = {(P[i] + P[j]) / 2, abs(P[i] - P[j]) / 2};
            for (int k = 0; k < j; k++) {
                if (C.inside(P[k])) continue;
                C.o = Center(P[i], P[j], P[k]);
                C.r = abs(C.o - P[i]);
            }
        }
    }
    return C;
}
```

4.9 旋轉卡尺

```
auto RotatingCalipers(const vector<Pt> &hull) { // 最遠點對 回傳距離平方
    int n = hull.size();
    auto ret = abs2(hull[0]);
    ret = 0;
    if (hull.size() <= 2) return abs2(hull[0] - hull[1]);
    for (int i = 0, j = 2; i < n; i++) {
        Pt a = hull[i], b = hull[(i + 1) % n];
        while(ori(hull[j], a, b) <
              (ori(hull[(j + 1) % n], a, b)))
            j = (j + 1) % n;
        chmax(ret, abs2(a - hull[j]));
        chmax(ret, abs2(b - hull[j]));
    }
    return ret;
}
```

5 Tree

5.1 is_anc

```
bool is_ancestor(int u,int v){
    return ( in[u] < in[v] && out[v] < out[u] );
}
```

5.2 LCA

```
int getlca(int x, int y) {
    if (is_ancestor(x, y)) return x;
    if (is_ancestor(y, x)) return y;
    for (int i = 19; i >= 0; i--) {
        if (!is_ancestor(anc[x][i], y))
            x = anc[x][i];
    }
    return anc[x][0];
}
```

5.3 倍增

```
int jump[N][logN];
for(int i = 1; i <= log2(N); i++){
    for(int now = 1; now <= N; now++){
        jump[now][i] = jump[jump[now][i - 1]][i - 1];
    }
}
//查詢
int query(int x,int k){
    int i = log2(k);
    while(k){
        if((1<<i) <= k){
            x = jump[x][i];
            k -= (1<<i);
        }
        i--;
    }
    return x;
}
```

5.4 tree_flaten

```
const int MAXN = 200005;
vector<int> G[MAXN];
int tin[MAXN], tout[MAXN], ord[MAXN], timer_;

void flatten(int root) {
    timer_ = 0;
    vector<pair<int,int>> st = {{root, 0}};   // (節點, 父節點)
    vector<int> it(MAXN, 0);
    while (!st.empty()) {
        auto [u, p] = st.back();
        if (it[u] == 0) tin[u] = ++timer_, ord[timer_] = u;
        if (it[u] < (int)G[u].size()) {
            int v = G[u][it[u]++];
            if (v != p) st.push_back({v, u});
        } else {
            tout[u] = timer_;
            st.pop_back();
        }
    }
}
```

6 DP

6.1 LIS

```
vector<int> dp;
for(int i = 0; i < n;++i){
    auto it = lower_bound(all(dp),A[i]) - dp.begin();
    if(it == dp.size()) dp.push_back(A[i]);
    else dp[it] = A[i];
}
cout << dp.size() << '\n';
```

6.2 LCS

```
for(int i = 1; i <= n; ++i){
    for(int j = 1; j <= m; ++j){
        dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        if(s[i] == t[j]) dp[i][j] = max(dp[i][j],dp[i-1][j-1]+1);
}}
```

6.3 拆分

```
//目前物品最多拿 k 個
int s = 0;
vector<pair<int,int>> A;//A是儲存所有捆
for(int r = 1; r * 2 <= k; r *= 2){//TLE的話先算k最大拿幾個
    s += r;
    A.push_back({r*c[i],r*v[i]});//r個一綑
}
A.push_back({(k-s)*c[i],(k-s)*v[i]});//最後一捆
```

6.4 編輯距離

```
for(int i = 1; i <= n; ++i)
    for(int j = 1; j <= m; ++j){
    if(s[i-1] == t[j-1]) dp[i][j] = dp[i-1][j-1];
    else {
        dp[i][j] = min({
            dp[i-1][j] + 1,
            dp[i][j-1] + 1,
            dp[i-1][j-1] + 1}
        );
    }
}
```

6.5 SOSDP

```
for(int i = 0;i < n; ++i)
    for (int s = 0; s < (1<<n) ; ++s)
        if (s & (1 << i))
            F[s] += F[s ^ (1 << i)];
//窮舉子集合
for(int s = mask; s ; s = (s-1)&mask;)
```

6.6 p_median

```
for(int h = 1; h <= k;++h){ //窮舉目前放了多少個區間
    for(int i = 1;i <= n;++i){ //第 h 個區間以 pi 為右界
        dp[h][i] = inf; //初始化
        for(int j = 1; j <= i; ++j){ //第 h 個區間的左界
            dp[h][i] = min(dp[h][i],dp[h-1][j-1] + d[j][i]);
        }
    }
}
cout << dp[k][n];
```

7 search

7.1 binary search

```
int bsearch_1(int l, int r)
{
    while (l < r)
    {
        int mid = (l + r) >> 1;
        if (check(mid)) r = mid;
        else l = mid + 1;
    }
    return l;
}
// ..............oooooooooo

int bsearch_2(int l, int r)
{
    while (l < r)
    {
        int mid = (l + r + 1) >> 1;
        if (check(mid)) l = mid;
        else r = mid - 1;
    }
    return l;
}
// oooooooooo..................
```

7.2 三分搜

```
int l = 1,r = 100;
while(l < r) {
    int lmid = l + (r - l) / 3; // l + 1/3區間大小
    int rmid = r - (r - l) / 3;  // r - 1/3區間大小
    lans = cal(lmid),rans = cal(rmid);
    // 求凹函數的極小值
    if(lans <= rans) r = rmid - 1;
    else l = lmid + 1;
}
```

8 Math

8.1 質數篩

```
/* 12721, 13331, 14341, 75577, 123457, 222557, 556679
* 999983, 1097774749, 1076767633, 100102021, 999997771
* 1001010013, 1000512343, 987654361, 999991231
* 999888733, 98789101, 987777733, 999991921, 1010101333
* 1010102101, 1000000000039, 1000000000000037
* 2305843009213693951, 4611686018427387847
* 9223372036854775783, 18446744073709551557 */
const int MXN = 1e8 + 50;
const int SQRTMXN = 1e4 + 50;
bitset<MXN> isprime;
vector<int> p(MXN, 0);
void sieve() {
    isprime[1] = 1;
    for (int i = 2; i <= SQRTMXN; i++) {
        if (!isprime[i])
            for (int j = i * i; j < MXN; j += i) {
                isprime[j] = 1;
                if(!p[j]) p[j] = i;
            }
    }
}
vector<int> factor(int x) {
```

```
    vector<int> fac{1};
    while (x > 1) {
        int prime = isprime[x] ? p[x] : x;
        int cnt = 0;
        while (x % prime == 0) {
            x /= prime;
            cnt++;
        }
        int sz = fac.size();
        int pw = 1;
        for (int k = 1; k <= cnt; k++) {
            pw *= prime;
            for (int i = 0; i < sz; i++)
                fac.push_back(fac[i] * pw);
        }
    }
    return fac;
}
```

8.2 fpow

```
int fpow(int a, int b){
    int ret = 1;
    for(;b; b>>=1, a=a*a%MOD) if(b&1) ret = ret*a%MOD;
    return ret;
}
```

8.3 exgcd

```
int exgcd(int a, int b, int &x, int &y) {
    if (b == 0) return x = 1, y = 0, a;
    int g = exgcd(b, a % b, y, x);
    y -= a / b * x;
    return g;
} // ax + by = gcd(a, b)
```

8.4 組合數

```
const int MXN = 1e6 + 5;
int fac[MXN], inv[MXN];
fac[0] = 1;
for(int i = 1 ; i < MXN ; i++) fac[i] = fac[i - 1] * i
    % MOD;
inv[MXN - 1] = fpow(fac[MXN - 1], MOD - 2);
for(int i = N - 2 ; i >= 0 ; i--) inv[i] = inv[i + 1] *
     (i + 1) % MOD;
int C(int n, int k){return fac[n] * inv[k] % MOD * inv[
    n - k] % MOD;}
```

8.5 josephus

```
int josephus(int n, int m){ //n人每m次
    int ans = 0;
    for (int i=1; i<=n; ++i)
        ans = (ans + m) % i;
    return ans;
}
```

8.6 josephus2

```
while(s < n){
        // ans = 0, s = 1
        int k = (s - ans - 1) / (m - 1) + 1;
        if(s + k > n) k = n - s;
        s += k;
        ans = (ans + k * m) % s;
}// n ¤j m ¤p
```

8.7 FFT

```
typedef long long ll;
typedef long double ld;
typedef complex<ld> cplx;
const int MAXN = 262144;          // 必須是 2^k，且 >=
    結果長度
const ld PI = acosl(-1);
const cplx I(0, 1);
cplx omega[MAXN + 1];
cplx arr[MAXN + 1];
// 使用前先呼叫一次
void pre_fft() {
    for (int i = 0; i <= MAXN; i++)
        omega[i] = exp(i * 2 * PI / MAXN * I);
}
// n 必須是 2^k，且 n <= MAXN
void fft(int n, cplx a[], bool inv = false) {
    int theta = MAXN / n;
    for (int m = n; m >= 2; m >>= 1) {
        int mh = m >> 1;
        for (int i = 0; i < mh; i++) {
            cplx w = omega[inv ? MAXN - (i * theta %
                MAXN)
                               : i * theta % MAXN];
            for (int j = i; j < n; j += m) {
                int k = j + mh;
                cplx x = a[j] - a[k];
                a[j] += a[k];
                a[k] = w * x;
            }
        }
        theta = (theta * 2) % MAXN;
    }
    int i = 0;
    for (int j = 1; j < n - 1; j++) {
        for (int k = n >> 1; k > (i ^= k); k >>= 1);
        if (j < i) swap(a[i], a[j]);
    }
    if (inv) for (i = 0; i < n; i++) a[i] /= n;
}
// ans = a * b，a 長度 _n，b 長度 _m，ans 長度 _n + _m
    - 1
void mul(int _n, const ll a[], int _m, const ll b[], ll
     ans[]) {
    int sum = _n + _m - 1, n = 1;
    while (n < sum) n <<= 1;
    assert(n <= MAXN);
    for (int i = 0; i < n; i++) {
        ld x = (i < _n ? a[i] : 0), y = (i < _m ? b[i]
            : 0);
        arr[i] = cplx(x + y, x - y);
    }
    fft(n, arr);
    for (int i = 0; i < n; i++) arr[i] = arr[i] * arr[i
        ];
    fft(n, arr, true);
    for (int i = 0; i < sum; i++)
        ans[i] = llroundl(arr[i].real() / 4);   // 負數
            係數也正確
}
int main() {
    pre_fft();
    ll a[] = {1, 2, 3};    // 1 + 2x + 3x^2
    ll b[] = {4, 5};       // 4 + 5x
    ll ans[4];
    mul(3, a, 2, b, ans);
    for (int i = 0; i < 4; i++) cout << ans[i] << ' ';
    // 4 13 22 15
}
```

8.8 Matrices_fpow

```
struct Mat {  // Mat t(r, c); 0-based
    ll a[200][200], r, c;
    Mat(int _r, int _c) : r(_r), c(_c) {
        memset(a, 0, sizeof(a));
    }
    void build() {  // 單位矩陣
        for (int i = 0; i < r; i++) a[i][i] = 1;
    }
};
Mat operator*(Mat x, Mat y) {
    Mat z(x.r, y.c);
    for (int i = 0; i < x.r; ++i)
        for (int j = 0; j < x.c; ++j)
            for (int k = 0; k < y.c; ++k)
                z.a[i][j] += x.a[i][k] * y.a[k][j]; //
                    如果要 MOD 可能要改一下這
    return z;
}
Mat fpow(Mat a, ll b) {
    Mat ret(a.r, a.c);
    ret.build();
    while (b) {
```

```
        if (b & 1) ret = ret * a;
        a = a * a;
        b >>= 1;
    }
    return ret;
}
```

8.9 Miller-Rabin

```
// n < 4,759,123,141        3 :  2, 7, 61
// n < 1,122,004,669,633    4 :  2, 13, 23, 1662803
// n < 3,474,749,660,383          6 :  pirmes <= 13
// n < 2^64                       7 :
// 2, 325, 9375, 28178, 450775, 9780504, 1795265022
// Make sure testing integer is in range [2, n-2] if
// you want to use magic.
typedef long long LL;
LL mul(LL a, LL b, LL m) { return (__int128)a * b % m;
    }
LL mypow(LL a, LL e, LL m) {
    LL r = 1; a %= m;
    while (e) {
        if (e & 1) r = mul(r, a, m);
        a = mul(a, a, m);
        e >>= 1;
    }
    return r;
}
const LL magic[] = {2, 325, 9375, 28178, 450775,
    9780504, 1795265022};
bool witness(LL a, LL n, LL u, int t) {
    if (!a) return 0;
    LL x = mypow(a, u, n);
    for (int i = 0; i < t; i++) {
        LL nx = mul(x, x, n);
        if (nx == 1 && x != 1 && x != n - 1) return 1;
        x = nx;
    }
    return x != 1;
}
bool miller_rabin(LL n) {
    if (n < 2) return 0;
    if (!(n & 1)) return n == 2;
    LL u = n - 1; int t = 0;
    while (!(u & 1)) u >>= 1, t++;
    int s = sizeof(magic) / sizeof(magic[0]);
    while (s--) {
        LL a = magic[s] % n;
        if (witness(a, n, u, t)) return 0;
    }
    return 1;
}
```

8.10 pollard rho

```
typedef long long LL;
LL mul(LL a, LL b, LL m) { return (__int128)a * b % m;
    }
LL add(LL a, LL b, LL m) { return ((__int128)a + b) % m
    ; }
// with mypow / witness / miller_rabin
LL f(LL x, LL c, LL mod) { return add(mul(x, x, mod), c
    , mod); }
LL pollard_rho(LL n) {
    LL c = 1, x = 0, y = 0, p = 2, q, t = 0;
    while (t++ % 128 || __gcd(p, n) == 1) {
        if (x == y) c++, y = f(x = 2, c, n);
        if ((q = mul(p, abs(x - y), n))) p = q;
        x = f(x, c, n); y = f(f(y, c, n), c, n);
    }
    return __gcd(p, n);}
```

9 String

9.1 KMP

```
//pi[i]=最大的k使得s[0...(k-1)] = s[i-(k-1)...i]
vector<int> prefunc(const string& s){
  int n = s.size();
  vector<int> pi(n);
  for(int i=1,j=0;i<n;++i){
    j = pi[i-1];
    while(j && s[j] != s[i]) j = pi[j-1];//取次小LCP
    if(s[j] == s[i]) ++j;
    pi[i] = j;
  }
  return pi;
}
//找s在str中出現的所有位子
vector<int> kmp(string str, string s) {
    vector<int> nxt = prefunc(s);
    vector<int> ans;
    for (int i = 0, j = 0; i < str.size(); i++) {
        while (j && str[i] != s[j]) j = nxt[j - 1];
        if (str[i] == s[j]) j++;
        if (j == s.size()) {
            ans.push_back(i - s.size() + 1);
            j = nxt[j - 1];
        }
    }
    return ans;
}
```

9.2 AC 自動機

```
struct ACautomata{
  struct Node{
    int cnt,i;
    Node *go[26], *fail, *dic;
    Node (){
      cnt = 0; fail = 0; dic = 0; i = 0;
      memset(go,0,sizeof(go));
    }
  }pool[1048576],*root;
  int nMem,n_pattern;
  Node* new_Node(){
    pool[nMem] = Node();
    return &pool[nMem++];
  }
  void init() {
    nMem=0;root=new_Node();n_pattern=0;
    add("");
  }
  void add(const string &str) { insert(root,str,0); }
  void insert(Node *cur, const string &str, int pos){
    for(int i=pos;i<str.size();i++){
      if(!cur->go[str[i]-'a'])
        cur->go[str[i]-'a'] = new_Node();
      cur=cur->go[str[i]-'a'];
    }
    cur->cnt++;
    // if (cur->i == 0) {cur->i = n_pattern;}
    // real_id[n_pattern] = cur->i;
    // n_pattern++;
    // 重複insert使用
    cur->i=n_pattern++;
  }
  void make_fail(){
    queue<Node*> que;
    que.push(root);
    while (!que.empty()){
      Node* fr=que.front(); que.pop();
      for (int i=0; i<26; i++){
        if (fr->go[i]){
          Node *ptr = fr->fail;
          while (ptr && !ptr->go[i]) ptr = ptr->fail;
          fr->go[i]->fail=ptr=(ptr?ptr->go[i]:root);
          fr->go[i]->dic=(ptr->cnt?ptr:ptr->dic);
          que.push(fr->go[i]);
  } } } }
  void query(string s){
      Node *cur=root;
      for(int i=0;i<(int)s.size();i++){
          while(cur&&!cur->go[s[i]-'a']) cur=cur->fail;
          cur=(cur?cur->go[s[i]-'a']:root);
          if(cur->i>=0) ans[cur->i]++;
          for(Node *tmp=cur->dic;tmp;tmp=tmp->dic)
              ans[tmp->i]++;
  } }// ans[i] : number of occurrence of pattern i
}AC;
//make_fail() before query()
```

9.3 minRotation

```
//rotate(begin(s),begin(s)+minRotation(s),end(s))
#define rep(i, s, e) for (int i = (s); i < (e); i++)
int minRotation(string s) {
  int a = 0, N = s.size(); s += s;
  rep(b,0,N) rep(k,0,N) {
    if(a+k == b || s[a+k] < s[b+k])
      {b += max(0, k-1); break;}
    if(s[a+k] > s[b+k]) {a = b; break;}
  } return a;
}
```

9.4 迴文樹

```
// state[i]以i為結尾的最長回文編號
// len[s]是對應的回文長度
// num[s]是有幾個回文後綴
// cnt[s]是這個回文子字串在整個字串中的出現次數
// fail[s]是他長度次長的回文後綴，aba的fail是a
// fac[i]前i個字元切成最少回文段所需的段數
const int MXN = 1000010;
struct PalT{
  int nxt[MXN][26],fail[MXN],len[MXN];
  int tot,lst,n,state[MXN],cnt[MXN],num[MXN];
  int diff[MXN],sfail[MXN],fac[MXN],dp[MXN];
  char s[MXN]={-1};
  int newNode(int l,int f){
    len[tot]=l,fail[tot]=f,cnt[tot]=num[tot]=0;
    memset(nxt[tot],0,sizeof(nxt[tot]));
    diff[tot]=(l>0?l-len[f]:0);
    sfail[tot]=(l>0&&diff[tot]==diff[f]?sfail[f]:f);
    return tot++;
  }
  int getfail(int x){
    while(s[n-len[x]-1]!=s[n]) x=fail[x];
    return x;
  }
  int getmin(int v){// 用到fac在抄
    dp[v]=fac[n-len[sfail[v]]-diff[v]];
    if(diff[v]==diff[fail[v]])
        dp[v]=min(dp[v],dp[fail[v]]);
    return dp[v]+1;
  }
  int push(){// 用到fac在抄
    int c=s[n]-'a',np=getfail(lst);
    if(!(lst=nxt[np][c])){
      lst=newNode(len[np]+2,nxt[getfail(fail[np])][c]);
      nxt[np][c]=lst; num[lst]=num[fail[lst]]+1;
    }
    fac[n]=n;
    for(int v=lst;len[v]>0;v=sfail[v])
        fac[n]=min(fac[n],getmin(v));
    return ++cnt[lst],lst;
  }
  void init(const char *_s){
    fac[0] = 0;
    tot=lst=n=0;
    newNode(0,1),newNode(-1,1);
    for(;_s[n];) s[n+1]=_s[n],++n,state[n-1]=push();
    for(int i=tot-1;i>1;i--) cnt[fail[i]]+=cnt[i];
  }
}palt;
```

9.5 Rolling Hash

```
struct RollingHash{
#define psz 2
    vector<int> primes={17, 75577};
    vector<int> MOD={998244353, 1000000007};
    vector<array<int, psz>> hash, base;
    void init(const string &s){
        hash.clear(); hash.resize(s.size());
        base.clear(); base.resize(s.size());
        for(int i=0;i<psz;i++){
            hash[0][i] = s[0];
            base[0][i] = 1;
        }
        for(int i=1;i<s.size();i++){
            for(int j=0;j<psz;j++){
                hash[i][j] = (hash[i-1][j] * primes[j]
                    % MOD[j] + s[i]) % MOD[j];
                base[i][j] = base[i-1][j] * primes[j] %
                     MOD[j];
            }
        }
    }
    array<int, psz> getHash(int l,int r){
        if(l == 0) return hash[r];
        array<int, psz> ret = hash[r];
        for(int i=0;i<psz;i++){
            ret[i] -= hash[l-1][i] * base[r-l+1][i] %
                MOD[i];
            if(ret[i]<0) ret[i]+=MOD[i];
        }
        return ret;
    }
}Hash;
```

9.6 SAIS

```
const int N = 300010;
struct SA{ // 將字串的所有後綴依字典序排序後的數組
#define REP(i,n) for ( int i=0; i<int(n); i++ )
#define REP1(i,a,b) for ( int i=(a); i<=int(b); i++ )
  bool _t[N*2];
  int _s[N*2], _sa[N*2], _c[N*2], x[N], _p[N], _q[N*2],
      hei[N], r[N];
  int operator [] (int i){ return _sa[i]; }
  void build(int *s, int n, int m){
    memcpy(_s, s, sizeof(int) * n);
    sais(_s, _sa, _p, _q, _t, _c, n, m);
    mkhei(n);
  }
  void mkhei(int n){
    REP(i,n) r[_sa[i]] = i;
    hei[0] = 0;
    REP(i,n) if(r[i]) {
      int ans = i>0 ? max(hei[r[i-1]] - 1, 0) : 0;
      while(_s[i+ans] == _s[_sa[r[i]-1]+ans]) ans++;
      hei[r[i]] = ans;
    }
  }
  void sais(int *s, int *sa, int *p, int *q, bool *t,
      int *c, int n, int z){
    bool uniq = t[n-1] = true, neq;
    int nn = 0, nmxz = -1, *nsa = sa + n, *ns = s + n,
        lst = -1;
#define MS0(x,n) memset((x),0,n*sizeof(*(x)))
#define MAGIC(XD) MS0(sa, n); \
    memcpy(x, c, sizeof(int) * z); \
    XD; \
    memcpy(x + 1, c, sizeof(int) * (z - 1)); \
    REP(i,n) if(sa[i] && !t[sa[i]-1]) sa[x[s[sa[i
        ]-1]]++] = sa[i]-1; \
    memcpy(x, c, sizeof(int) * z); \
    for(int i = n - 1; i >= 0; i--) if(sa[i] && t[sa[i
        ]-1]) sa[--x[s[sa[i]-1]]] = sa[i]-1;
    MS0(c, z);
    REP(i,n) uniq &= ++c[s[i]] < 2;
    REP(i,z-1) c[i+1] += c[i];
    if (uniq) { REP(i,n) sa[--c[s[i]]] = i; return; }
    for(int i = n - 2; i >= 0; i--) t[i] = (s[i]==s[i
        +1] ? t[i+1] : s[i]<s[i+1]);
    MAGIC(REP1(i,1,n-1) if(t[i] && !t[i-1]) sa[--x[s[i
        ]]]=p[q[i]=nn++]=i);
    REP(i, n) if (sa[i] && t[sa[i]] && !t[sa[i]-1]) {
      neq=lst<0||memcmp(s+sa[i],s+lst,(p[q[sa[i]]+1]-sa
          [i])*sizeof(int));
      ns[q[lst=sa[i]]]=nmxz+=neq;
    }
    sais(ns, nsa, p + nn, q + n, t + n, c + z, nn, nmxz
        + 1);
    MAGIC(for(int i = nn - 1; i >= 0; i--) sa[--x[s[p[
        nsa[i]]]]] = p[nsa[i]]);
  }
}sa;
int H[ N ], SA[ N ];
void suffix_array(int* ip, int len) {
  // should padding a zero in the back
  // ip is int array, len is array length
  // ip[0..n-1] != 0, and ip[len] = 0
  ip[len++] = 0;
  sa.build(ip, len, 128);
  for (int i=0; i<len; i++) {
    H[i] = sa.hei[i + 1];
```

```
    SA[i] = sa._sa[i + 1];
    // SA[i] = 第i小的後綴是第幾個字元開始的後綴字串
    // H[i] = lcp(s(sa[i]: ), s(sa[i - 1]: ))
  }
  // resulting height, sa array \in [0,len)
}
```

9.7 trie

```
struct trie{
    trie *nxt[26];//01trie改2
    int cnt; //紀錄有多少個字串以此節點結尾
    int sz;  //有多少字串的前綴包括此節點
    trie():cnt(0),sz(0){
        memset(nxt,0,sizeof(nxt));
    }
};
trie *root = new trie();
  void insert(string& s){
    trie *now = root;
    for(auto i:s){//01trie改高位元往下
        now->sz++;
        if(now->nxt[i-'a'] == NULL){
            now->nxt[i-'a'] = new trie();
        }
        now = now->nxt[i-'a'];
    }
    now->cnt++;
    now->sz++;
}
int query_prefix(string& s){
    trie *now = root;
    for(auto i:s){
        if(now->nxt[i-'a'] == NULL){
            return 0;
        }
        now = now->nxt[i-'a'];
    }
    return now->sz;
}
int query_count(string& s){
    trie *now = root;
    for(auto i:s){
        if(now->nxt[i-'a'] == NULL){
            return 0;
        }
        now = now->nxt[i-'a'];
    }
    return now->cnt;
}
```

9.8 z_value

```
int z[MAXN];
void Z_value(const string& s) {
    //z[i] = lcp(s[1...],s[i...])
  int i, j, left, right, len = s.size();
  left=right=0; z[0]=len;
  for(i=1;i<len;i++) {
    j=max(min(z[i-left],right-i),0);
    for(;i+j<len&&s[i+j]==s[j];j++);
    z[i]=j;
    if(i+z[i]>right) {
      right=i+z[i];
      left=i;
}   }   }
```

9.9 馬拉車

```
//以每個字元為中心的最長迴文長度
//abc -> @a@b@c
void z_value_pal(char* s, int len, int* z) {
    len = (len << 1) + 1;
    for (int i = len - 1; i >= 0; i--)
        s[i] = i & 1 ? s[i >> 1] : '@';
    z[0] = 1;
    for (int i = 1, l = 0, r = 0; i < len; i++) {
        z[i] = i < r ? min(z[l + l - i], r - i) : 1;
        while (i - z[i] >= 0 && i + z[i] < len && s[i -
             z[i]] == s[i + z[i]])
            ++z[i];
        if (i + z[i] > r)
            l = i, r = i + z[i];
    }
}
```

10 好東西

10.1 大悲咒

千手千眼無礙大悲心陀羅尼南無喝囉怛那哆囉夜耶・南無阿唎耶，婆盧羯帝爍缽囉耶・菩提薩埵婆耶・摩訶薩埵婆耶・摩訶迦盧尼迦耶・唵，薩皤囉罰曳・數怛那怛寫・南無悉吉慄埵伊蒙阿唎耶・婆盧吉帝室佛囉楞馱婆・南無那囉謹墀・醯利摩訶皤哆沙咩・

薩婆阿他豆輸朋，阿逝孕，薩婆薩哆那摩婆薩哆，那摩婆伽，摩罰特豆・怛姪他・唵，阿婆盧醯・盧迦帝・迦羅帝・夷醯唎・摩訶菩提薩埵，薩婆薩婆・摩囉摩囉，摩醯摩醯唎馱孕・俱盧俱盧羯蒙・度盧度盧罰闍耶帝・摩訶罰闍耶帝・陀囉陀囉・地唎尼・室佛囉耶・

遮囉遮囉・摩麼罰摩囉・穆帝隸・伊醯伊醯・室那室那・阿囉參佛囉舍利・罰沙罰參・佛囉舍耶・呼嚧呼嚧摩囉・呼嚧呼嚧醯利・娑囉娑囉，悉唎悉唎・蘇嚧蘇嚧・

菩提夜菩提夜・菩馱夜菩馱夜・彌帝唎夜・那囉謹墀・地利瑟尼那・波夜摩那・娑婆訶・悉陀夜・娑婆訶・摩訶悉陀夜・娑婆訶・悉陀喻藝・室皤囉耶・娑婆訶・那囉謹墀・娑婆訶・摩囉那囉・娑婆訶・悉囉僧阿穆佉耶，娑婆訶・娑婆摩訶阿悉陀夜・娑婆訶・

者吉囉阿悉陀夜・娑婆訶・波陀摩羯悉陀夜・娑婆訶・那囉謹墀皤伽囉耶・娑婆訶・摩婆利勝羯囉夜・娑婆訶・南無喝囉怛那哆囉夜耶，南無阿唎耶・婆嚧吉帝・爍皤囉夜・娑婆訶・唵，悉殿都・漫多囉・跋陀耶，娑婆訶・

10.2 心經

觀自在菩薩（かんじざいぼさつ / Kan-jizai bosatsu）
行深般若波羅蜜多時(ぎょうじんはんにゃはらみたじ / Gyō jin hannya haramita ji)
照見五蘊皆空（しょうけんごうんかいくう / Shō ken go-un kai kū）
度一切苦厄（どいっさいくやく / Do is-sai ku yaku）
舍利子（しゃりし / Sharishi）
色不異空、空不異色（しきふいくう、くうふいしき / Shiki fu-i kū, kū fu-i shiki）
色即是空、空即是色（しきそくぜくう、くうそくぜしき / Shiki soku ze kū, kū soku ze shiki）
受想行識、亦復如是（じゅそうぎょうしき、やくぶにょぜ / Ju sō gyō shiki, yaku bu nyo ze）
舍利子、是諸法空相（しゃりし、ぜしょほうくうそう / Sharishi, ze sho-hō kū sō）
不生不滅、不垢不淨、不增不減（ふしょうふめつ、ふくふじょう、ふぞうふげん / Fu shō fu metsut, fu ku fu jō, fu zō fu gen）
是故空中、無色、無受想行識（ぜこくうちゅう、むしき、むじゅそうぎょうしき / Ze ko kū chū, mu shiki, mu ju sō gyō shiki）
無眼耳鼻舌身意（むげんにびぜつしんい / Mu gen ni bi zetsu shin i）
無聲香味觸法（むしょうこうみそくほう / Mu shō kō mi soku hō）
無眼界、乃至無意識界（むげんかい、ないしむいしきかい / Mu gen kai, naishi mu ishiki kai）
無無明、亦無無明盡（むむみょう、やくむむみょうじん / Mu mumyō, yaku mu mumyō jin）
乃至無老死、亦無老死盡（ないしむろうし、やくむろうしじん / Naishi mu rōshi, yaku mu rōshi jin）
無苦集滅道（むくしゅうめつどう / Mu ku shū metsudō）
無智亦無得（むちやくむとく / Mu chi yaku mu toku）
以無所得故、菩提薩埵（いむしょとくこ、ぼだいさた / I mu sho toku ko, bodaisatta）
依般若波羅蜜多故、心無罣礙（えはんにゃはらみたこ、しんむけいげ / E hannya haramita ko, shin mu keige）
無罣礙故、無有恐怖（むけいげこ、むうくふ / Mu keige ko, mu ū ku fu）
遠離一切顛倒夢想、涅槃究竟（おんりいっさいてんどうむそう、ねはんくうきょう / Onri is-sai tendō musō, nehan kūkyō）
三世諸佛、依般若波羅蜜多故、得阿耨多羅三藐三菩提（さんぜしょぶつ、えはんにゃはらみたこ、とくあのくたらさんみゃくさんぼだい / Sanze shobutsu, e hannya haramita ko, toku anokutara sammyaku sambodai）
故知、般若波羅蜜多、是大神咒、是大明咒、是無上咒、是無等等咒、能除一切苦、真實不虛（こち、はんにゃはらみた、ぜだいじんしゅ、ぜだいみょうしゅ、ぜむじょうしゅ、ぜむとうどうしゅ、のうじょいっさいく、しんじつふこ / Ko chi, hannya haramita, ze dai jin shu, ze dai myō shu, ze mujō shu, ze mutōdō shu, nō jo is-sai ku, shinjitsu fu ko）
故說、般若波羅蜜多咒、即說咒曰（こせつ、はんにゃはらみたしゅ、そくせつしゅわつ / Ko setsu, hannya haramita shu, soku setsu shu watsu）：
羯諦　羯諦　波羅羯諦　波羅僧羯諦　菩提薩婆訶（ぎゃてい　ぎゃてい　はらぎゃてい　はらそうぎゃてい　ぼじそわか / Gyatei gyatei hara-gyatei hara-sōgyatei boji sowaka）
般若心經（はんにゃしんぎょう / Hannya shingyō）



